

# Agents

Intelligent agents are considered by many to be the ultimate goal of AI. The classic book by Stuart Russell and Peter Norvig, *Artificial Intelligence: A Modern Approach* (Prentice Hall, 1995) defines the field of *artificial intelligence research* as “the study and design of rational agents.”

The unprecedented capabilities of foundation models have opened the door to agentic applications that were previously unimaginable. These new capabilities make it finally possible to develop autonomous, intelligent agents to act as our assistants, coworkers, and coaches. They can help us create a website, gather data, plan a trip, do market research, manage a customer account, automate data entry, prepare us for interviews, interview our candidates, negotiate a deal, etc. The possibilities seem endless, and the potential economic value of these agents is enormous.



AI-powered agents are an emerging field, with no established theoretical frameworks for defining, developing, and evaluating them. This section is a best-effort attempt to build a framework from the existing literature, but it will evolve as the field does. Compared to the rest of the book, this section is more experimental.

This section will start with an overview of agents, and then continue with two aspects that determine the capabilities of an agent: tools and planning. Agents, with their new modes of operations, have new modes of failures. This section will end with a discussion on how to evaluate agents to catch these failures.

Even though agents are novel, they are built upon concepts that have already appeared in this book, including self-critique, chain-of-thought, and structured outputs.

## Agent Overview

The term *agent* has been used in many different engineering contexts, including but not limited to a software agent, intelligent agent, user agent, conversational agent, and reinforcement learning agent. So, what exactly is an agent?

An agent is anything that can perceive its environment and act upon that environment.<sup>10</sup> This means that an agent is characterized by the *environment* it operates in and *the set of actions* it can perform.

The *environment* an agent can operate in is defined by its use case. If an agent is developed to play a game (e.g., *Minecraft*, *Go*, *Dota*), that game is its environment. If you want an agent to scrape documents from the internet, the environment is the internet. If your agent is a cooking robot, the kitchen is its environment. A self-driving car agent's environment is the road system and its adjacent areas.

The *set of actions* an AI agent can perform is augmented by the *tools* it has access to. Many generative AI-powered applications you interact with daily are agents with access to tools, albeit simple ones. ChatGPT is an agent. It can search the web, execute Python code, and generate images. RAG systems are agents, and text retrievers, image retrievers, and SQL executors are their tools.

There's a strong dependency between an agent's environment and its set of tools. The environment determines what tools an agent can potentially use. For example, if the environment is a chess game, the only possible actions for an agent are the valid chess moves. However, an agent's tool inventory restricts the environment it can operate

---

<sup>10</sup> *Artificial Intelligence: A Modern Approach* (1995) defines an agent as anything that can be viewed as perceiving its environment through sensors and acting upon that environment through actuators.

in. For example, if a robot's only action is swimming, it'll be confined to a water environment.

Figure 6-8 shows a visualization of SWE-agent (Yang et al., 2024), an agent built on top of GPT-4. Its environment is the computer with the terminal and the file system. Its set of actions include navigate repo, search files, view files, and edit lines.

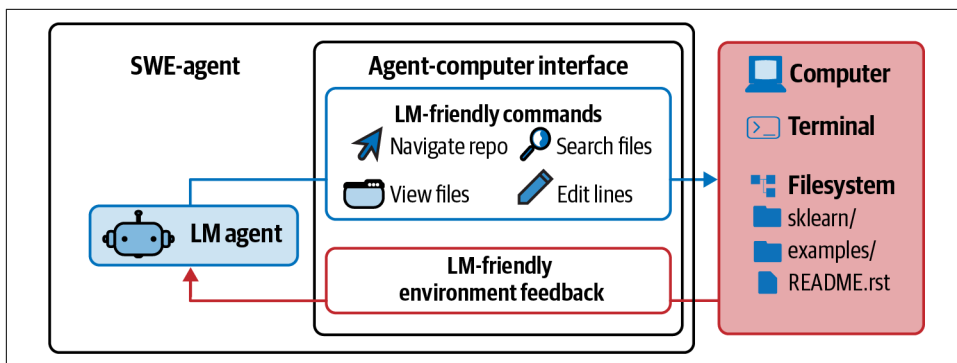


Figure 6-8. SWE-agent (Yang et al., 2024) is a coding agent whose environment is the computer and whose actions include navigation, search, and editing. Adapted from an original image licensed under CC BY 4.0.

An AI agent is meant to accomplish tasks typically provided by the users in the inputs. In an AI agent, AI is the brain that processes the information it receives, including the task and feedback from the environment, plans a sequence of actions to achieve this task, and determines whether the task has been accomplished.

Let's get back to the RAG system with tabular data in the Kitty Vogue example. This is a simple agent with three actions: response generation, SQL query generation, and SQL query execution. Given the query "Project the sales revenue for Fruity Fedora over the next three months", the agent might perform the following sequence of actions:

1. Reason about how to accomplish this task. It might decide that to predict future sales, it first needs the sales numbers from the last five years. Note that the agent's reasoning is shown as its intermediate response.
2. Invoke SQL query generation to generate the query to get sales numbers from the last five years.
3. Invoke SQL query execution to execute this query.
4. Reason about the tool outputs and how they help with sales prediction. It might decide that these numbers are insufficient to make a reliable projection, perhaps because of missing values. It then decides that it also needs information about past marketing campaigns.

5. Invoke SQL query generation to generate the queries for past marketing campaigns.
6. Invoke SQL query execution.
7. Reason that this new information is sufficient to help predict future sales. It then generates a projection.
8. Reason that the task has been successfully completed.

Compared to non-agent use cases, agents typically require more powerful models for two reasons:

- Compound mistakes: an agent often needs to perform multiple steps to accomplish a task, and the overall accuracy decreases as the number of steps increases. If the model’s accuracy is 95% per step, over 10 steps, the accuracy will drop to 60%, and over 100 steps, the accuracy will be only 0.6%.
- Higher stakes: with access to tools, agents are capable of performing more impactful tasks, but any failure could have more severe consequences.

A task that requires many steps can take time and money to run.<sup>11</sup> However, if agents can be autonomous, they can save a lot of human time, making their costs worthwhile.

Given an environment, the success of an agent in an environment depends on the tool inventory it has access to and the strength of its AI planner. Let’s start by looking into different kinds of tools a model can use.

## Tools

A system doesn’t need access to external tools to be an agent. However, without external tools, the agent’s capabilities would be limited. By itself, a model can typically perform one action—for example, an LLM can generate text, and an image generator can generate images. External tools make an agent vastly more capable.

Tools help an agent to both perceive the environment and act upon it. Actions that allow an agent to perceive the environment are *read-only actions*, whereas actions that allow an agent to act upon the environment are *write actions*.

This section gives an overview of external tools. How tools can be used will be discussed in “[Planning](#)” on page 281.

The set of tools an agent has access to is its tool inventory. Since an agent’s tool inventory determines what an agent can do, it’s important to think through what and

---

<sup>11</sup> A complaint in the early days of agents is that agents are only good for burning through your API credits.

how many tools to give an agent. More tools give an agent more capabilities. However, the more tools there are, the more challenging it is to understand and utilize them well. Experimentation is necessary to find the right set of tools, as discussed in [“Tool selection” on page 295](#).

Depending on the agent’s environment, there are many possible tools. Here are three categories of tools that you might want to consider: knowledge augmentation (i.e., context construction), capability extension, and tools that let your agent act upon its environment.

### **Knowledge augmentation**

I hope that this book, so far, has convinced you of the importance of having the relevant context for a model’s response quality. An important category of tools includes those that help augment your agent’s knowledge of your agent. Some of them have already been discussed: text retriever, image retriever, and SQL executor. Other potential tools include internal people search, an inventory API that returns the status of different products, Slack retrieval, an email reader, etc.

Many such tools augment a model with your organization’s private processes and information. However, tools can also give models access to public information, especially from the internet.

Web browsing was among the earliest and most anticipated capabilities to be incorporated into chatbots like ChatGPT. Web browsing prevents a model from going stale. A model goes stale when the data it was trained on becomes outdated. If the model’s training data was cut off last week, it won’t be able to answer questions that require information from this week unless this information is provided in the context. Without web browsing, a model won’t be able to tell you about the weather, news, upcoming events, stock prices, flight status, etc.

I use web browsing as an umbrella term to cover all tools that access the internet, including web browsers and specific APIs such as search APIs, news APIs, GitHub APIs, or social media APIs such as those of X, LinkedIn, and Reddit.

While web browsing allows your agent to reference up-to-date information to generate better responses and reduce hallucinations, it can also open up your agent to the cesspools of the internet. Select your Internet APIs with care.

### **Capability extension**

The second category of tools to consider are those that address the inherent limitations of AI models. They are easy ways to give your model a performance boost. For example, AI models are notorious for being bad at math. If you ask a model what is 199,999 divided by 292, the model will likely fail. However, this calculation is trivial if

the model has access to a calculator. Instead of trying to train the model to be good at arithmetic, it's a lot more resource-efficient to just give the model access to a tool.

Other simple tools that can significantly boost a model's capability include a calendar, timezone converter, unit converter (e.g., from lbs to kg), and translator that can translate to and from the languages that the model isn't good at.

More complex but powerful tools are code interpreters. Instead of training a model to understand code, you can give it access to a code interpreter so that it can execute a piece of code, return the results, or analyze the code's failures. This capability lets your agents act as coding assistants, data analysts, and even research assistants that can write code to run experiments and report results. However, automated code execution comes with the risk of code injection attacks, as discussed in [“Defensive Prompt Engineering” on page 235](#). Proper security measurements are crucial to keep you and your users safe.

External tools can make a text-only or image-only model multimodal. For example, a model that can generate only texts can leverage a text-to-image model as a tool, allowing it to generate both texts and images. Given a text request, the agent's AI planner decides whether to invoke text generation, image generation, or both. This is how ChatGPT can generate both text and images—it uses DALL-E as its image generator. Agents can also use a code interpreter to generate charts and graphs, a LaTeX compiler to render math equations, or a browser to render web pages from HTML code.

Similarly, a model that can process only text inputs can use an image captioning tool to process images and a transcription tool to process audio. It can use an OCR (optical character recognition) tool to read PDFs.

*Tool use can significantly boost a model's performance compared to just prompting or even finetuning.* Chameleon (Lu et al., 2023) shows that a GPT-4-powered agent, augmented with a set of 13 tools, can outperform GPT-4 alone on several benchmarks. Examples of tools this agent used are knowledge retrieval, a query generator, an image captioner, a text detector, and Bing search.

On ScienceQA, a science question answering benchmark, Chameleon improves the best published few-shot result by 11.37%. On TabMWP (Tabular Math Word Problems) (Lu et al., 2022), a benchmark involving tabular math questions, Chameleon improves the accuracy by 17%.

## Write actions

So far, we've discussed read-only actions that allow a model to read from its data sources. But tools can also perform write actions, making changes to the data sources. A SQL executor can retrieve a data table (read) but can also change or delete the table